

Программирование на С

Лекция 10.0.5

Кэш

Одна задача — два обхода

По строкам

```
long long sum = 0;

for (size_t i = 0; i < n; ++i)
    for (size_t j = 0; j < n; ++j)
        sum += a[i * n + j];
```

По столбцам

```
long long sum = 0;

for (size_t j = 0; j < n; ++j)
    for (size_t i = 0; i < n; ++i)
        sum += a[i * n + j];
```

Одинаковая матрица. Одинаковое число элементов. Одинаковое число сложений.

Будет ли разница во времени?

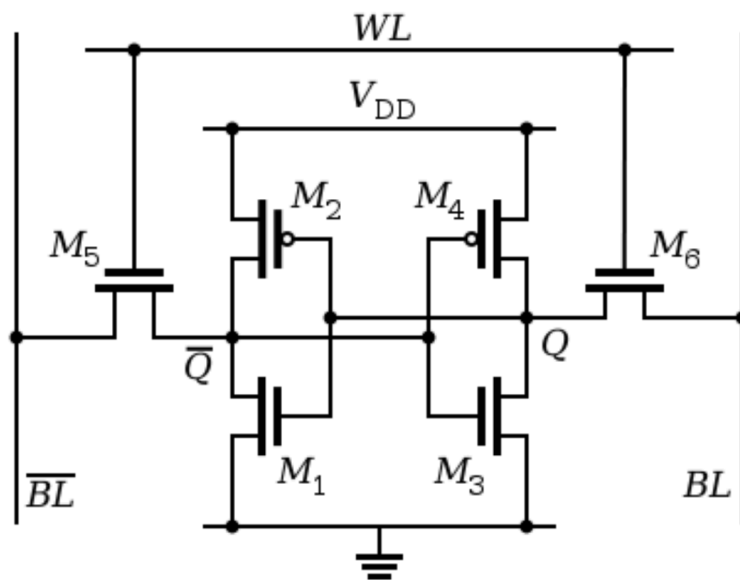
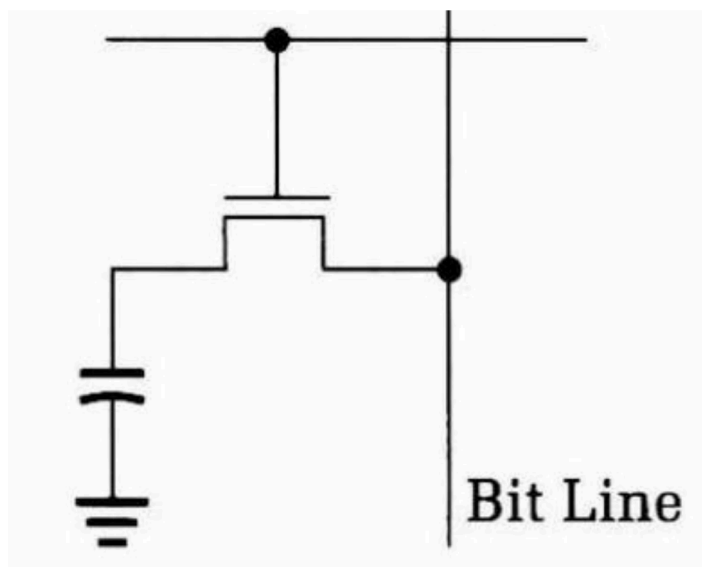
Проверим

Matrix	row-major, ms	column-major, ms	ratio

Арифметика почти та же.

Время выполнения — нет.

Как хранится один бит?



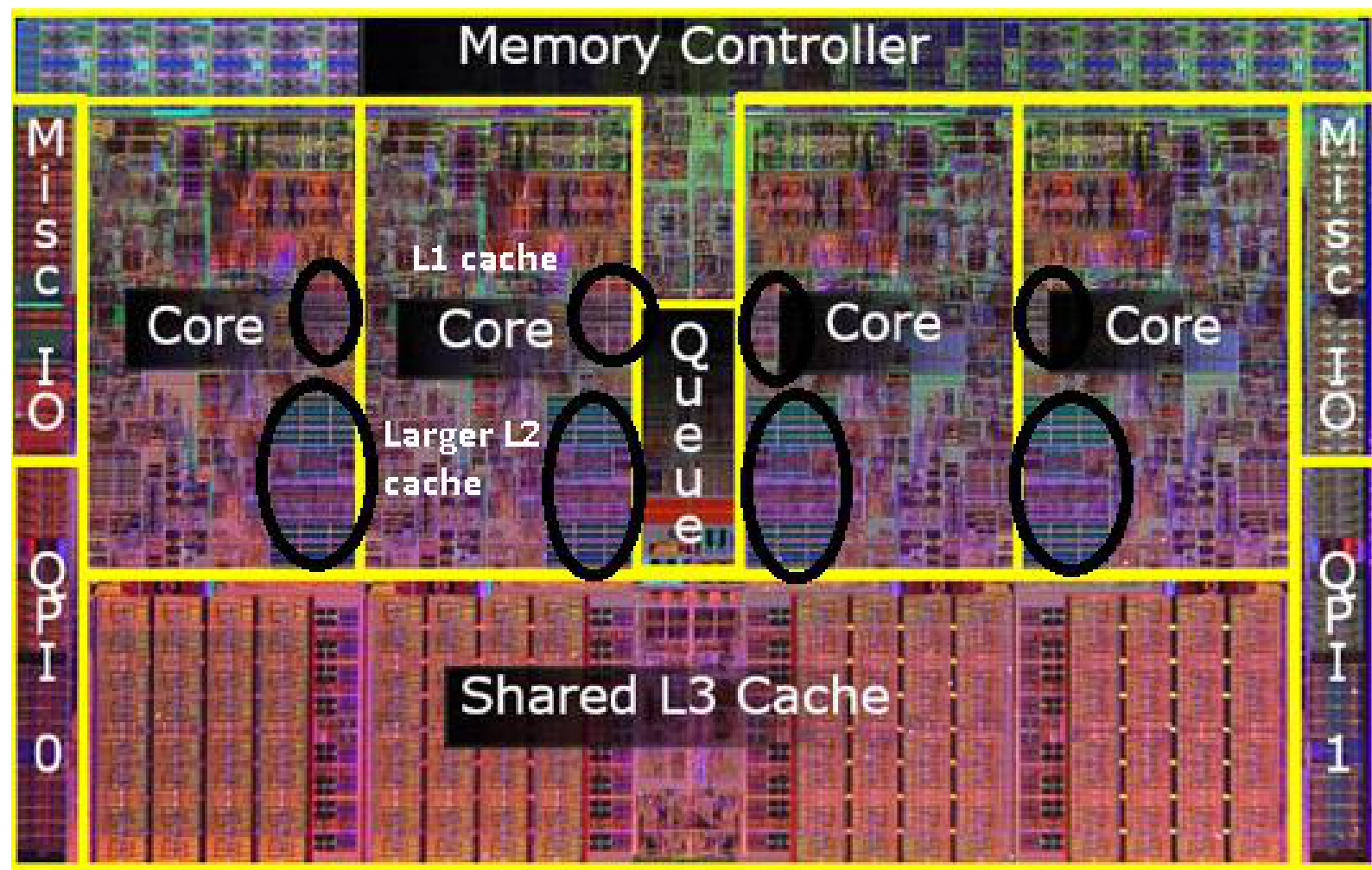
Кэш — это
память на
процессоре

Кэш

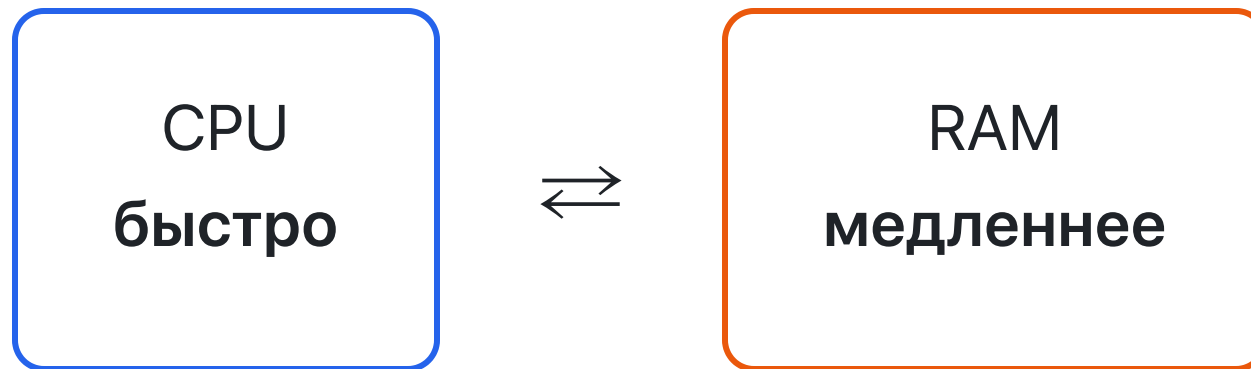
L1

L2

L3



Процессор и память работают с разной скоростью



Процессор может ждать данные дольше, чем выполнять операции над ними.

Почему не сделать всю память быстрой?

	DRAM	SRAM
Ячейка	проще	сложнее
Плотность	выше	ниже
Стоимость	ниже	выше
Скорость	ниже	выше
Использование	RAM	cache

Большая память → DRAM

Маленькая быстрая память рядом с CPU → SRAM

Добавим кэш



Кэш хранит копии данных, которые могут снова понадобиться процессору.

Кэш работает блоками

RAM ... | a[0] | a[1] | a[2] | a[3] | a[4] | ... | a[15] | ...



64 B (cache line)

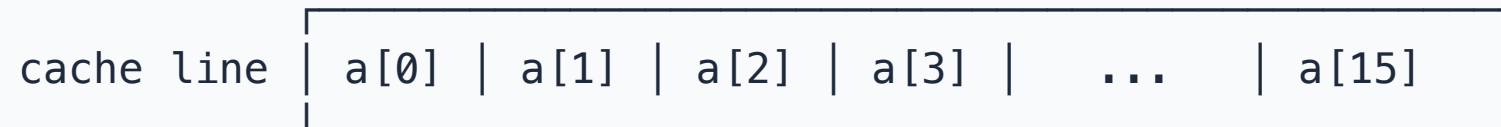
↓ копируется целиком ↓

CACHE | a[0] | a[1] | a[2] | ... | a[15] |

Почему это работает?

```
int x = a[0];
```

cache line



Теперь:

```
int y = a[1]; // cash hit!!!
```

Почему это работает?

cache hit

Нужные данные уже есть в кэше.

cache miss

Нужной линии нет — приходится
обращаться дальше.

Иерархия памяти



← быстрее

← меньше

больше →

медленнее →

Вернёмся к матрице



По строкам

```
for (size_t i = 0; i < n; ++i)
    for (size_t j = 0; j < n; ++j)
        sum += a[i * n + j];
```

Память читается почти
последовательно.

По столбцам

```
for (size_t j = 0; j < n; ++j)
    for (size_t i = 0; i < n; ++i)
        sum += a[i * n + j];
```

Между соседними обращениями: $n * \text{sizeof}(\text{int})$ байт.

Локальность данных

Последовательный доступ

много данных из одной cache line ->
-> много cache hits

Большие скачки

часто нужна новая cache line -
>
-> больше cache misses

Порядок доступа к данным может влиять на скорость не меньше, чем количество операций.

Программирование на С

Лекция 10

Память

Три вида памяти

```
int x[100];

int main(void)
{
    int y[100] = {0};

    int *pz = calloc(100, sizeof(int));

    free(pz);
}
```

Где находятся массивы `x`, `y` и память,
на которую указывает `pz`?

Где располагаются данные?

```
int x[100];

int main(void)
{
    int y[100] = {0};

    int *pz = calloc(100, sizeof(int));

    free(pz);
}
```

Программа

Статическая память
x

Куча (Heap)
память для pz

Стек (Stack)
y

Три области памяти

Область	Что хранится
Стек	локальные переменные
Куча	динамическая память
Статическая память	глобальные и <code>static</code> переменные

Время жизни объекта

Время жизни объекта — это промежуток времени, в течение которого объект существует в памяти.

Область видимости

```
int main(void)
{
    int x = 10;

    {
        int y = 20;
    }

    printf("%d", y);
}
```

Почему программа не компилируется?

Область видимости переменной в for

Переменную-счётчик можно объявить прямо в заголовке цикла:

```
for (int i = 0; i < 10; i++) {  
    printf("%d\n", i);  
}
```

```
printf("%d\n", i); // ошибка
```

Время жизни $\neq$ область видимости

Объект может

- существовать;
- не существовать;

и одновременно быть

- доступным;
- недоступным.

Это разные свойства объекта.

Класс хранения

Класс хранения определяет свойства объекта:

- время жизни;
- область видимости;
- компоновку.

Каждый объект может иметь только один класс хранения.

Класс хранения `auto`

```
int main(void)
{
    int x = 10;
}
```

Все локальные переменные по умолчанию имеют класс хранения `auto`.

Явное указание auto

```
int main(void)
{
    auto int x = 10;
}
```

Программа полностью эквивалентна

```
int main(void)
{
    int x = 10;
}
```

Ключевое слово **auto** почти никогда не пишут.

Свойства auto

Свойство	Значение
Расположение	стек
Время жизни	до выхода из блока
Область видимости	текущий блок

`auto` в C23

Начиная с C23, `auto` можно использовать для автоматического определения типа переменной:

```
auto x = 10;  
auto y = 3.14;  
auto p = &x;
```

Компилятор определяет тип по инициализатору:

```
int x = 10;  
double y = 3.14;  
int *p = &x;
```

При использовании `auto` для определения типа переменная должна иметь инициализатор.

`auto` не делает язык динамически типизированным: тип определяется во время компиляции.

Что произойдёт?

```
int *f(void)
{
    int x = 10;

    return &x;
}
```

Корректна ли эта программа?

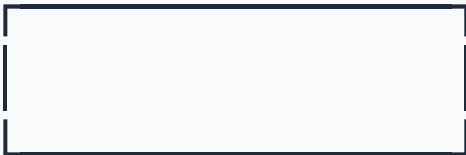
После выхода из функции

Вход

$x = 10$

↓

Выход

An empty rectangular box with a black border, intended for the output value after the function call.

Можно ли сохранить переменную?

Иногда необходимо, чтобы локальная переменная существовала между вызовами функции.

Для этого используется класс хранения `static`.

Класс хранения static

Ключевое слово `static` изменяет время жизни объекта.

Объект создаётся один раз и существует до завершения программы.

Локальная переменная static

```
void f(void) {  
    static int x = 0;  
    x++;  
    printf("%d\n", x);  
}
```

Что будет выведено?

```
void f(void) {  
    static int x = 0;  
    x++;  
    printf("%d\n", x);  
}
```

```
f();  
f();  
f();
```

?

Ответ

```
void f(void) {  
    static int x = 0;  
    x++;  
    printf("%d\n", x);  
}
```

```
f();  
f();  
f();
```

```
1  
2  
3
```

Переменная создаётся только один раз.

После выхода из функции она не уничтожается.

auto и static

	auto	static
Создаётся при входе в блок	✓	✗
Уничтожается при выходе	✓	✗
Существует всю программу	✗	✓

static не делает переменную глобальной

```
void f(void)
{
    static int x = 0;
}

int main(void)
{
    x++;
}
```

Почему программа не компилируется?

static не делает переменную глобальной

```
void f(void)
{
    static int x = 0;
}

int main(void)
{
    x++;
}
```

static изменяет время жизни объекта,

но не делает локальную переменную доступной вне функции.

Глобальная переменная

```
int x = 10;  
  
int main(void)  
{  
    printf("%d\n", x);  
}
```

Свойства глобальной переменной

Свойство	Значение
Расположение	статическая память
Время жизни	вся программа
Область видимости	весь файл

Область видимости

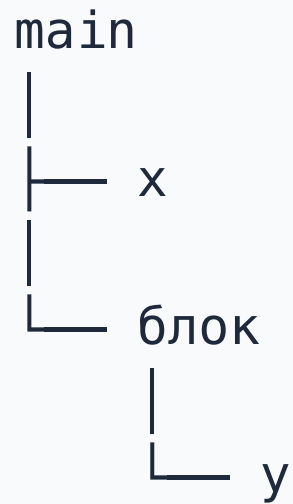
```
int main(void)
{
    int x = 10;

    {
        int y = 20;
    }

    printf("%d\n", x);
    printf("%d\n", y);
}
```

Почему программа не компилируется?

Переменная доступна только внутри блока



После выхода из блока имя **y** становится недоступным.

Переменные с одинаковыми именами

```
int main(void)
{
    int x = 10;

    {
        int x = 20;
        printf("%d\n", x);
    }

    printf("%d\n", x);
}
```

Что будет выведено?

```
int x = 10;  
  
{  
    int x = 20;  
    printf("%d\n", x);  
}  
  
printf("%d\n", x);
```

?

Ответ

```
int x = 10;

{
    int x = 20;
    printf("%d\n", x);
}

printf("%d\n", x);
```

20
10

Внутренняя переменная скрывает внешнюю, пока выполняется её блок.

Следует ли использовать одинаковые имена?

Лучше избегать повторного объявления переменных с одинаковыми именами.

Сравнение переменных

Переменная	Где хранится	Время жизни	Область видимости
Локальная (<code>auto</code>)	стек	блок	блок
Локальная (<code>static</code>)	статическая память	вся программа	блок
Глобальная	статическая память	вся программа	файл

Передача аргументов

При вызове функции значения аргументов копируются в её параметры.

Передача по значению

```
void inc(int x) {  
    x++;  
}  
  
int main(void) {  
    int a = 5;  
  
    inc(a);  
  
    printf("%d\n", a);  
}
```

Что будет выведено?

Передача указателя

```
void inc(int *x) {  
    (*x)++;  
}  
  
int main(void) {  
    int a = 5;  
  
    inc(&a);  
  
    printf("%d\n", a);  
}
```

Что будет выведено?

Передача "по указателю"

При вызове функции

```
inc(&a);
```

в параметр функции копируется адрес переменной.

В языке C все аргументы передаются по значению.

Даже если значением является адрес.

Указатель тоже копируется

```
void f(int *p) {  
    p = NULL;  
}  
  
int main(void) {  
    int a = 5;  
  
    int *q = &a;  
  
    f(q);  
}
```

После вызова функции `q == NULL` или `q == &a` ?

Константность в языке **C**

Ключевое слово `const` обозначает, что объект нельзя изменять.

```
int a = 4;  
int const b = 5;
```

```
a = 6;    // допустимо  
b = 6;    // ошибка
```

`const` добавляет к типу ограничение на изменение объекта.

Зачем нужна константность?

`const` позволяет явно указать намерение программиста:

- значение не должно изменяться;
- функция не должна изменять переданные данные;
- случайное изменение будет обнаружено компилятором.

Используйте `const`, если объект не должен изменяться.

Объявление неизменяемого объекта

Следующие объявления эквивалентны:

```
int const x = 5;  
const int y = 6;
```

И `x`, и `y` имеют тип «неизменяемое целое».

```
x = 10;    // ошибка  
y = 10;    // ошибка
```

Инициализация const

Неизменяемому объекту обычно задают значение при объявлении:

```
int const size = 100;
```

После инициализации изменить его нельзя:

```
size = 200; // ошибка
```

const не означает, что значение обязательно известно во время компиляции.

Указатель на неизменяемые данные

```
int a = 5;  
int b = 8;  
  
int const *p = &a;
```

Через `p` можно прочесть значение:

```
printf("%d\n", *p);
```

Но нельзя изменить его:

```
*p = 10; // ошибка
```

Сам указатель можно изменить

```
int a = 5;  
int b = 8;  
  
int const *p = &a;  
  
p = &b;    // допустимо
```

Сначала:

$p \longrightarrow a$

Затем:

$p \longrightarrow b$

const относится к объекту, на который указывает **p**.

Данные могут измениться другим способом

```
int a = 5;  
int const *p = &a;  
  
a = 10;    // допустимо
```

Теперь

```
printf("%d\n", *p);
```

Выведет

10

Указатель **p** запрещает изменять объект через **p**, но не делает сам объект **a** неизменяемым.

Неизменяемый указатель

```
int a = 5;  
int b = 8;  
  
int *const p = &a;
```

Через `p` можно изменить объект:

```
*p = 10; // допустимо
```

Но нельзя изменить сам указатель:

```
p = &b; // ошибка
```

Что именно неизменно?

```
int *const p = &a;
```

p → a
x

Указатель всегда должен указывать на **a**.

Но значение **a** можно изменить:

```
*p = 10;
```

const относится к самому указателю **p**.

Неизменяемый указатель на неизменяемые данные

```
int const *const p = &a;
```

Нельзя изменить данные через указатель:

```
*p = 10; // ошибка
```

Нельзя изменить сам указатель:

```
p = &b; // ошибка
```

Четыре варианта указателей

Объявление	Можно изменить <code>*p</code>	Можно изменить <code>p</code>
<code>int *p</code>	✓	✓
<code>int const *p</code>	✗	✓
<code>int *const p</code>	✓	✗
<code>int const *const p</code>	✗	✗

Как читать объявление?

Начинаем с имени переменной:

```
int const *p;
```

`p` — указатель на неизменяемое целое.

```
int *const p = &a;
```

`p` — неизменяемый указатель на целое.

```
int const *const p = &a;
```

`p` — неизменяемый указатель на неизменяемое целое.

Положение const

Модификатор `const` относится к тому, что находится слева от него:

```
int const x;  
int *const p = &x;
```

Если слева ничего нет, он относится к тому, что справа:

```
const int x;
```

East const и west const

```
int const x;    // east const  
const int y;    // west const
```

Обе формы допустимы и означают одно и то же.

const и typedef

```
typedef int *int_ptr;
```

Теперь

```
int_ptr const p = &a;
```

означает

```
int *const p = &a;
```

const делает неизменяемым весь тип, заданный через **typedef**.

const в параметрах функций

```
void print_array(int const *arr, int size)
{
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
}
```

Функция может читать элементы
массива:

```
printf("%d", arr[i]);
```

Но не может изменять их:

```
arr[i] = 0; // ошибка
```

Функция, изменяющая массив

```
void clear_array(int *arr, int size)
{
    for (int i = 0; i < size; i++)
        arr[i] = 0;
}
```

Параметр

```
int *arr
```

позволяет функции изменять элементы массива.

Функция, только читающая массив

```
void print_array(int const *arr, int size);
```

const в типе параметра показывает, что функция не изменяет элементы массива через этот параметр.

Это делает назначение функции понятнее и защищает данные от случайного изменения.

Две формы записи параметра

В параметрах функции следующие записи эквивалентны:

```
void print_array(int const *arr, int size);
```

```
void print_array(int const arr[], int size);
```

В обоих случаях функция получает указатель на первый элемент массива.

Что выбрать?

Функция изменяет объект:

```
void sort(int *arr, int size);
```

Функция только читает объект:

```
void print_array(int const *arr, int size);
```

Добавляйте `const` к параметру-указателю, если функция не должна изменять данные.

Как читать сложные объявления?

В языке **C** встречаются объявления, которые сложно прочесть с первого взгляда.

Например:

```
char *const *(*next)(int *const);
```

Не пытайтесь читать такие объявления слева направо.

Начинайте с имени переменной.

Утилита cdecl

Программа `cdecl` умеет объяснять объявления на языке C.

Например:

```
cdecl> explain char *const *(*next)(int *const)
```

Результат:

```
declare next as pointer to function  
(pointer to const int)  
returning pointer to const pointer to char
```

**Даже опытные программисты
используют `cdecl`,
когда встречаются очень сложные
объявления.**

Пример

```
char *const *(*next)(int *const);
```

Начинаем с имени: `next`

Далее читаем наружу:

- `(*next)` — указатель;
- `(...)(int *const)` — на функцию;
- функция принимает неизменяемый указатель на `int`;
- функция возвращает указатель;
- на неизменяемый указатель;
- на `char`.

Упражнение

Попробуйте прочитать объявление самостоятельно.

```
char *(*carr[10])(int **);
```

Что представляет собой переменная `carr`?

Ответ

```
char *(*carr[10])(int **);
```

Начинаем с имени: `carr`

Далее:

- `carr[10]` — массив из 10 элементов;
- `(*...)` — каждый элемент является указателем;
- `(...)(int **)` — на функцию;
- функция принимает `int **`;
- функция возвращает `char *`.

Упражнение

Попробуйте прочитать объявление самостоятельно.

```
unsigned (*search)(const int *[2], unsigned[][4]);
```

Что представляет собой переменная `search` ?

Ответ

```
unsigned (*search)(const int *[2], unsigned[][4]);
```

Начинаем с имени: `search`

Далее:

- `(*search)` — указатель;
- `(...)(...)` — на функцию;
- первый параметр — массив из двух указателей на неизменяемые `int`;
- второй параметр — двумерный массив `unsigned` с четырьмя столбцами;
- функция возвращает `unsigned`.